

VOLUME 1

Start Without Hesitation

*Project setup, engineering rhythm, Git basics, and the first working
React screen*

A beginner-to-engineer rebuild guide grounded in the real repository

PATTERN > PRACTICE > REPETITION > REFINEMENT > HABIT

Learning Map

Use this volume as a build session, not passive reading. Type the examples, pause at checkpoints, and keep the architecture questions beside your editor.

Chapter	You will learn
1. The engineer's starting ritual	Turn an idea into responsibilities and a tiny first milestone.
2. Set up Macfolio	Create the Vite project, install dependencies, and understand configuration.
3. Naming and placement	Use predictable names and decide where code belongs.
4. Build the first vertical slice	Render a shell, component, data object, and style together.
5. Practice loop	Debug, commit, document, and repeat with confidence.

Chapter 1: The Engineer's Starting Ritual

Chapter outcome: You will be able to start a new idea with a one-page design instead of an empty-editor panic.

WHAT A starting ritual is a repeatable set of questions asked before files are created.

WHY It changes uncertainty into named responsibilities and keeps the first version small.

WHEN Use it for every new project and every feature that touches more than one file.

WHERE Write the answers in a README, issue, or architecture note before implementation.

HOW Define the user, outcome, entities, responsibilities, dependencies, and smallest end-to-end slice.

- User: who is trying to accomplish something?
- Outcome: what observable result proves the feature works?
- Entities: what nouns carry data, such as window, project, file, or route?
- Responsibilities: which pieces load data, store state, decide behavior, and render UI?
- Dependencies: which browser APIs, packages, and remote services are involved?
- Growth question: what becomes painful if the feature grows ten times?

Macfolio example: The product is not merely a page. It is a portfolio presented through two interaction shells: a macOS-like desktop and an iPhone-like mobile experience. Both consume portfolio content, but each owns its navigation model.

Three Ways To Understand It

Beginner explanation: Planning is writing down what each helper is responsible for before coding it.

Developer explanation: Planning defines contracts between UI, state, and data modules.

Engineering explanation: Planning reduces coupling by making ownership and dependency direction explicit before implementation pressure blurs them.

Chapter 2: Set Up Macfolio

Chapter outcome: You will understand every setup file and be able to reproduce the development environment from an empty folder.

1. Install a current Node.js LTS release and Git.
2. Create a project with Vite's React template.
3. Install the libraries listed in package.json.
4. Create .env.local with VITE_GITHUB_USERNAME for public GitHub data.
5. Run the development server and open the local address shown in the terminal.

The minimum setup loop

```

npm create vite@latest macfolio -- --template react
cd macfolio
npm install
npm run dev

```

Package	Version	Why Macfolio uses it
@gsap/react	^2.1.2	React lifecycle integration for GSAP.
@monaco-editor/react	^4.7.0	VS Code-like source preview.
@tailwindcss/vite	^4.2.2	Tailwind integration for Vite.
clsx	^2.1.1	Conditional class names.
dayjs	^1.11.20	Compact date and time formatting.
framer-motion	^12.40.0	Mobile screen transitions and gestures.
gsap	^3.15.0	Window drag and animation mechanics.
immer	^11.1.4	Ergonomic immutable updates through Zustand middleware.
lucide-react	^1.7.0	Consistent interface icons.
postcss-nesting	^14.0.0	Supporting project dependency; see its imports in the repository atlas.
prismjs	^1.30.0	Supporting project dependency; see its imports in the repository atlas.
react	^19.2.4	Component and hook runtime.
react-dom	^19.2.4	Mount React in the browser.
react-icon	^1.0.0	Supporting project dependency; see its imports in the repository atlas.
react-icons	^5.6.0	File-type icons.
react-pdf	^10.4.1	Render the resume PDF in a window.
react-responsive	^10.0.1	Desktop/mobile shell selection.
react-syntax-highlighter	^16.1.1	Supporting project dependency; see its imports in the repository atlas.
react-tooltip	^5.30.0	Supporting project dependency; see its imports in the repository atlas.
tailwindcss	^4.2.2	Utility classes alongside authored CSS.

Package	Version	Why Macfolio uses it
zustand	^5.0.12	Small domain stores for shared state.

Configuration Files

File	Responsibility	Do not put here
index.html	Browser document and #root mount point.	React feature markup.
src/main.jsx	Create the React root and load global CSS.	Business logic or window behavior.
vite.config.js	Plugins and # path aliases.	Runtime feature settings.
eslint.config.js	Code-quality rules.	Formatting-only preferences.
jsconfig.json	Editor path resolution.	Build behavior.
package-lock.json	Exact dependency graph for reproducible installs.	Hand-edited dependency choices.

Actual application entry point

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './app/App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

ReactDOM.createRoot finds the div with id root, creates React's rendering boundary, and renders App inside StrictMode. StrictMode intentionally repeats some development-only lifecycle work to expose unsafe side effects; it does not double-render production output.

Chapter 3: Naming and Placement

Chapter outcome: You will choose file, component, hook, function, constant, and CSS names that reveal ownership.

Thing	Convention	Macfolio example
-------	------------	------------------

Thing	Convention	Macfolio example
React component	PascalCase	FinderWindow.jsx
Hook	camelCase beginning with use	usePortfolioFileSystem.js
Utility	verb or decision phrase	buildWorkLocation.js
Constant collection	UPPER_SNAKE_CASE	WINDOW_CONFIG
Feature folder	lowercase kebab-case	code-preview/
CSS class	lowercase kebab-case with domain prefix	mobile-app-frame

Placement rule: Put code beside the feature that owns its change. Move it to shared only after multiple unrelated features genuinely need the same behavior.

- A component renders and coordinates UI.
- A hook coordinates reusable React state or effects.
- A utility performs a pure transformation or decision.
- A store owns state shared across distant parts of one domain.
- A data file contains stable content or configuration, not active behavior.
- An API module speaks to an external service and returns plain data.

Chapter 4: Build the First Vertical Slice

Chapter outcome: You will build one visible path through data, logic, UI, and CSS before expanding the project.

A vertical slice crosses every required layer

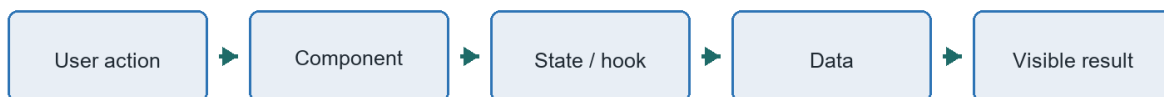


Figure 1. A small end-to-end path gives faster feedback than building every folder first.

A deliberately small first slice

```
const apps = [{ id: 'finder', name: 'Finder', icon: '/images/finder.png' }];

function Dock() {
  return apps.map((app) => (
    <button key={app.id} onClick={() => openWindow(app.id)}>
      <img src={app.icon} alt="" />
      <span>{app.name}</span>
    </button>
  ));
}
```

The array is data, Dock is presentation, and openWindow is behavior owned elsewhere. That separation is already architecture, even before the project becomes large.

Chapter 5: Practice Loop

Chapter outcome: You will use a calm repeatable cycle for building, checking, documenting, and improving features.

6. Make the smallest observable change.
7. Read the browser and terminal feedback.
8. Explain the execution flow in plain language.
9. Run lint and a production build.
10. Commit one coherent idea with a message that describes the outcome.
11. Record architecture decisions and lessons when the reasoning will matter later.

A compact verification and commit rhythm

```
npm run lint
npm run build
git status
git add <files>
git commit -m "Build desktop window opening flow"
```

Checkpoint

- Beginner: Explain why package-lock.json is committed.
- Intermediate: Design a folder structure for a notes app with offline storage.
- Advanced: Explain which project decisions deserve an ADR and which do not.

Architecture Review

Dimension	Score	Reason
Architecture	8/10	The current app has clear feature boundaries, with a few compatibility exports left during migration.
Maintainability	8/10	Purpose headers and docs make ownership visible; global CSS remains broad.

Dimension	Score	Reason
Scalability	7/10	Client-side public APIs fit a portfolio but would need a server boundary for private or high-volume data.
Readability	8/10	Explicit names and small coordinators make most flows approachable.
Reusability	7/10	The project correctly favors local ownership over premature shared abstractions.